

Assignment - 1

Program - 1 - Design and implementation of an AND Gate using VHDL

⇒ Inputs : A, B

⇒ Output : C

⇒ Truth table :

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

⇒ Program :

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity and_gate is
```

```
    Port (
```

```
        A : in  STD_LOGIC;  -- Input A
```

```
        B : in  STD_LOGIC;  -- Input B
```

```
        C : out STD_LOGIC   -- Output C = A AND B
```

```
    );
```

```
end and_gate;
```

```
architecture Behavioral of and_gate is
```

```
begin
```

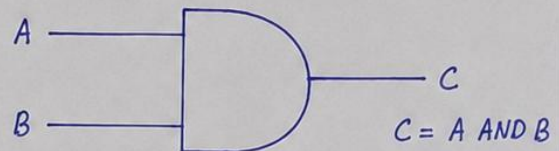
```
    -- Concurrent signal assignment:
```

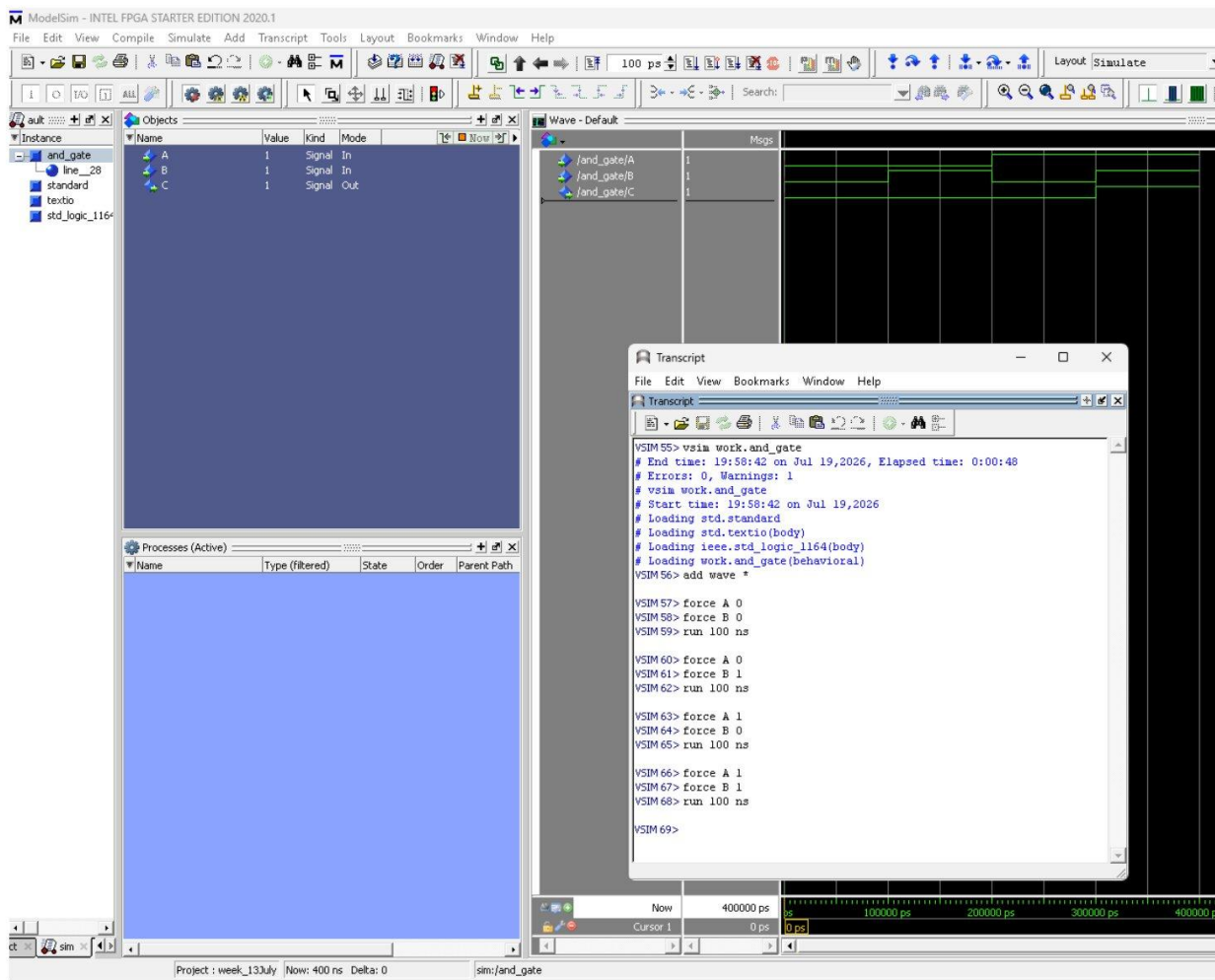
```
    -- C gets the logical AND of A and B
```

```
    C <= A AND B;
```

```
end Behavioral;
```

Circuit :





ModelSim Simulation – AND Gate

Assignment - 1

Program - 1 - Design and implementation of an OR Gate using VHDL

⇒ Inputs : A, B

⇒ Output : C

⇒ Truth table :

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

⇒ Program :

Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity or_gate is

Port (

A : in STD_LOGIC; -- Input A

B : in STD_LOGIC; -- Input B

C : out STD_LOGIC; -- Output C = A OR B

);

end or_gate;

architecture Behavioral of or_gate is

begin

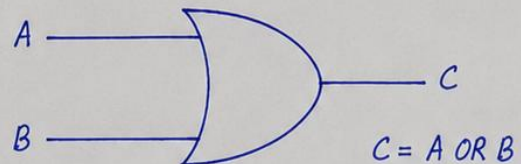
-- Concurrent signal assignment:

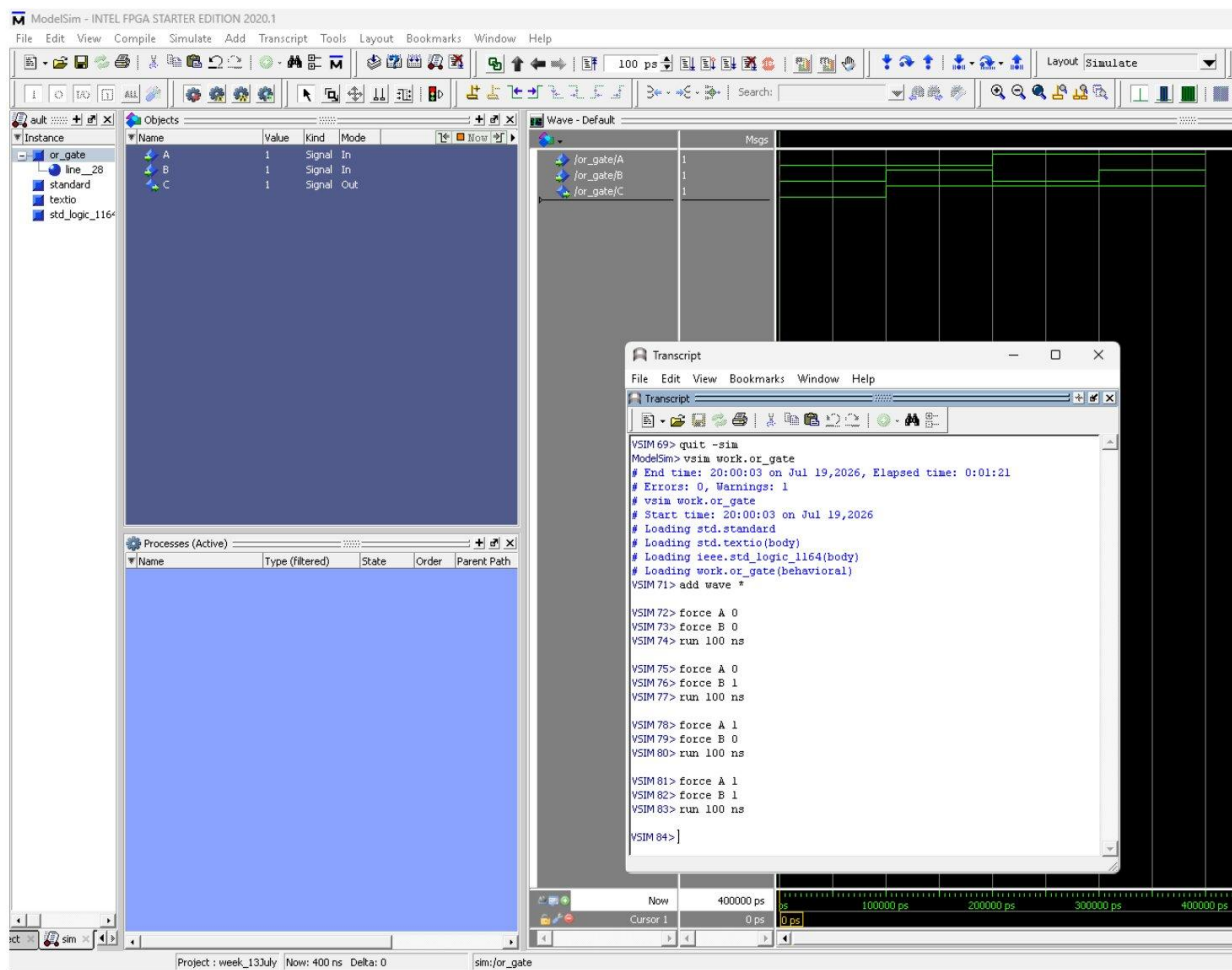
-- C gets the logical OR of A and B

C <= A OR B;

end Behavioral;

Circuit :





ModelSim Simulation – OR Gate

Assignment - 1

Program - 1 - Design and implementation of a NOT Gate using VHDL

⇒ Inputs : A

⇒ Output : C

⇒ Truth table :

A	C
0	1
1	0

⇒ Program :

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity not_gate is

Port (

A : in STD_LOGIC; -- Input signal

C : out STD_LOGIC -- Output signal (inverted)

);

end not_gate;

architecture Behavioral of not_gate is

begin

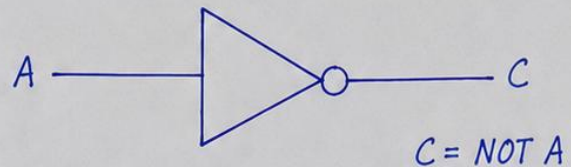
-- Concurrent signal assignment:

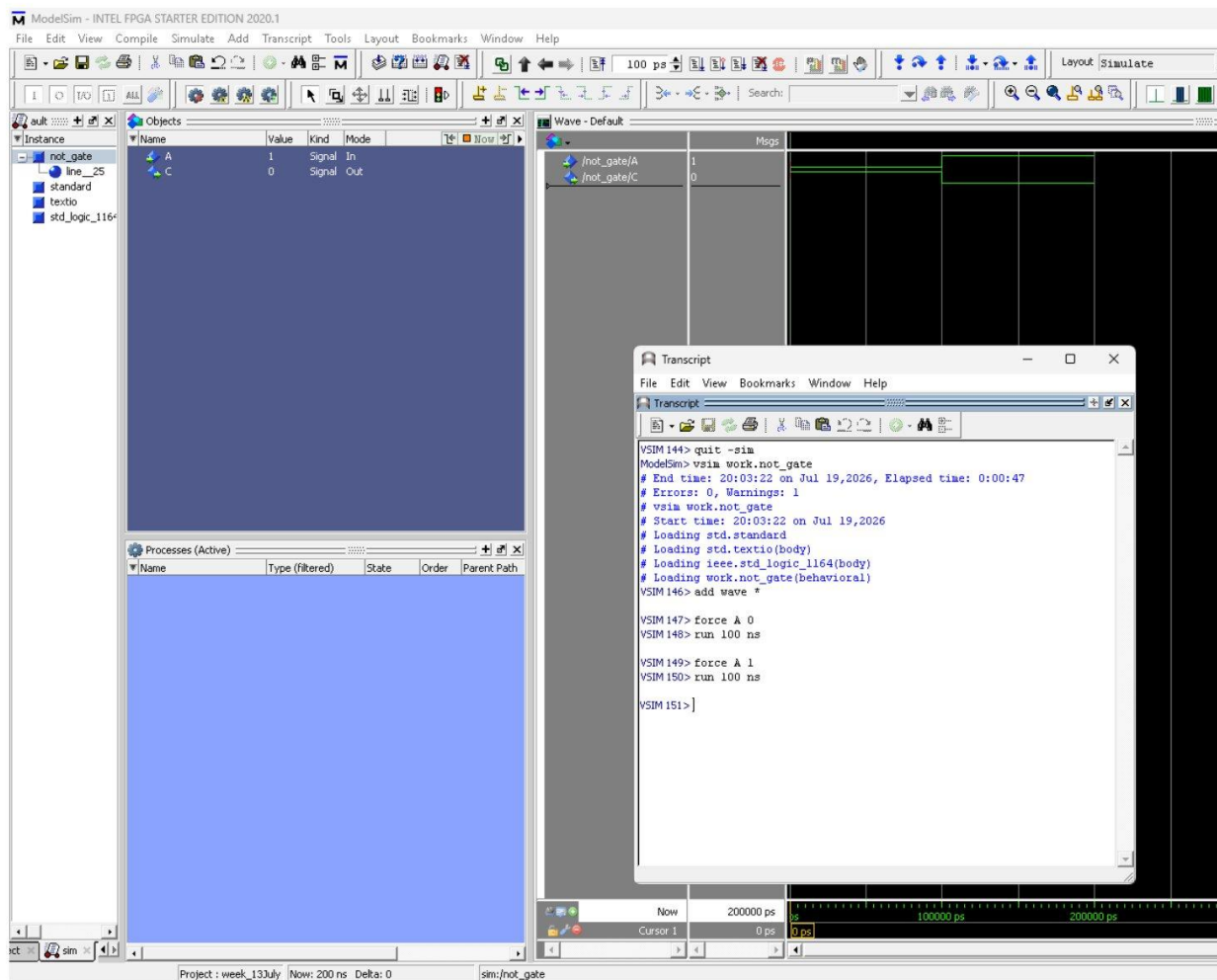
-- C gets the logical NOT of A

C <= NOT A;

end Behavioral;

Circuit :





ModelSim Simulation – NOT Gate

Assignment - 1

Program - 1 - Design and implementation of a NAND Gate using VHDL

⇒ Inputs : A, B

⇒ Output : C

⇒ Truth table :

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

⇒ Program :

Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity nand_gate is

Port (

A : in STD_LOGIC; -- Input A

B : in STD_LOGIC; -- Input B

C : out STD_LOGIC; -- Output C = A NAND B

);

end nand_gate;

architecture Behavioral of nand_gate is

begin

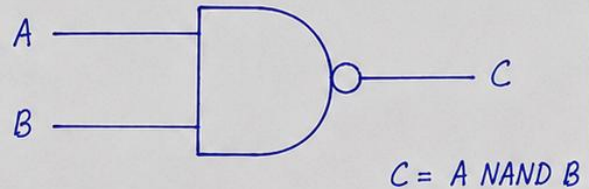
-- Concurrent signal assignment:

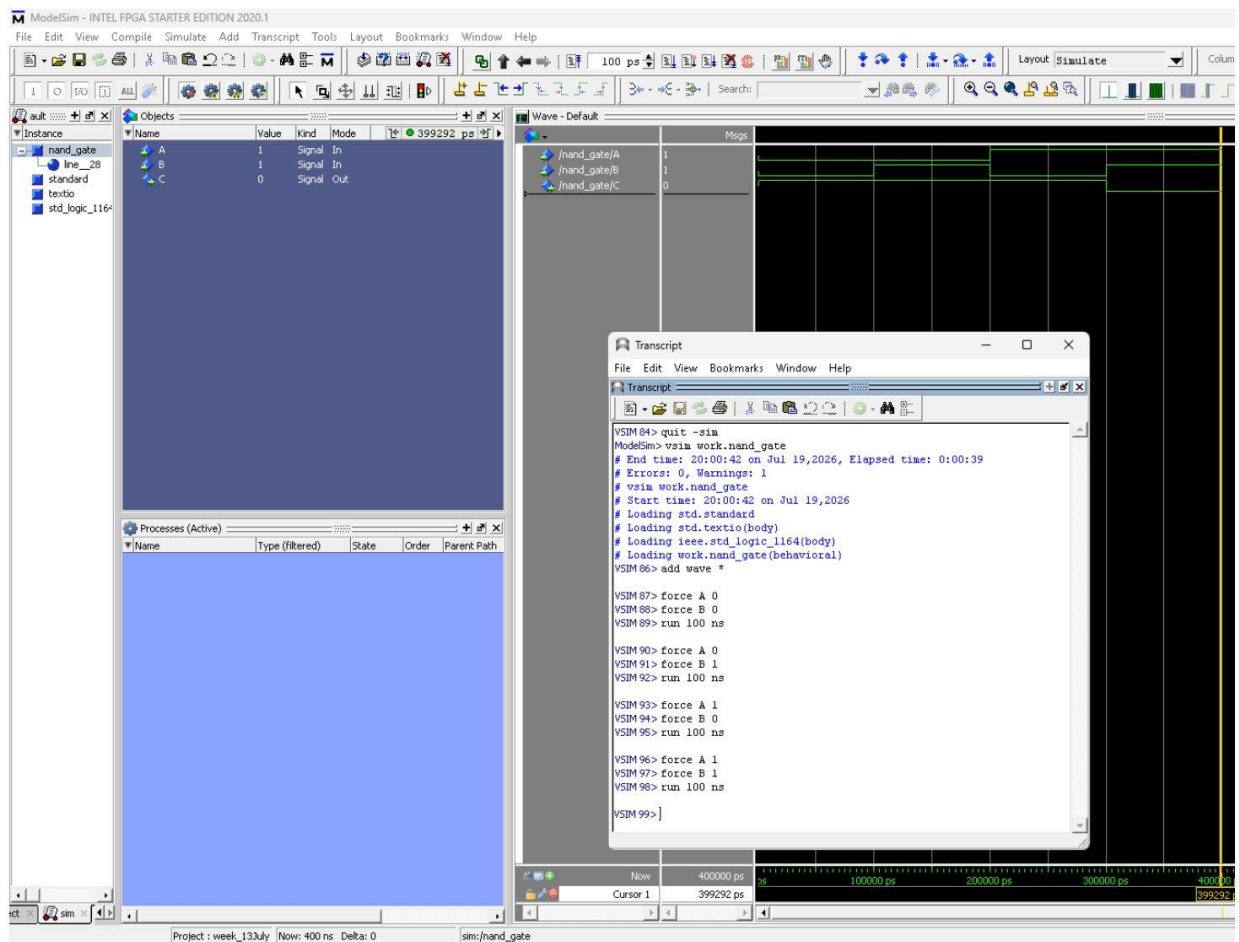
-- C gets the logical NAND of A and B

C <= A NAND B;

end Behavioral;

Circuit :





ModelSim Simulation – NAND Gate

Assignment - 1

Program - 1 - Design and implementation of a NOR Gate using VHDL

⇒ Inputs : A, B

⇒ Output : C

⇒ Truth table :

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

⇒ Program :

Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity nor_gate is

Port (

A : in STD_LOGIC; -- Input A

B : in STD_LOGIC; -- Input B

C : out STD_LOGIC; -- Output C = A NOR B

);

end nor_gate;

architecture Behavioral of nor_gate is

begin

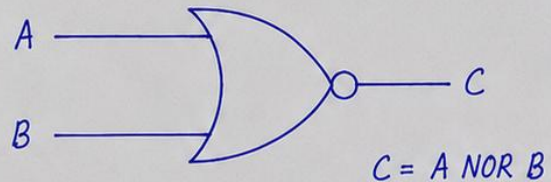
-- Concurrent signal assignment:

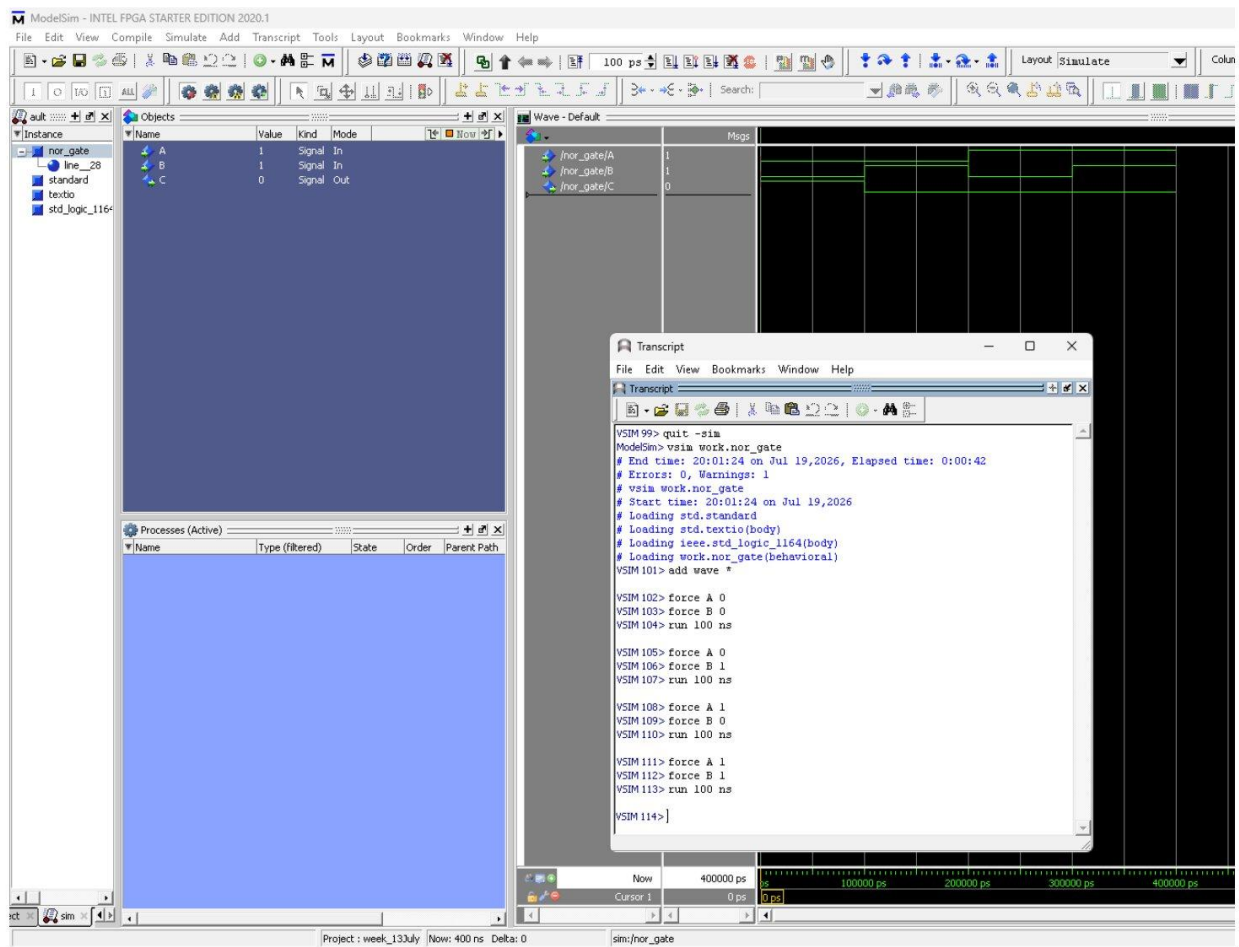
-- C gets the logical NOR of A and B

C <= A NOR B;

end Behavioral;

Circuit :





ModelSim Simulation – NOR Gate

Assignment - 1

Program - 1 - Design and implementation of an XOR Gate using VHDL

⇒ Inputs : A, B

⇒ Output : C

⇒ Truth table :

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

⇒ Program :

Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity xor_gate is

Port (

A : in STD_LOGIC; -- Input A

B : in STD_LOGIC; -- Input B

C : out STD_LOGIC; -- Output C = A XOR B

);

end xor_gate;

architecture Behavioral of xor_gate is

begin

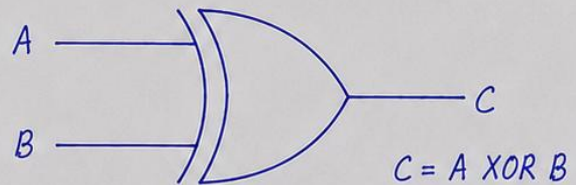
-- Concurrent signal assignment:

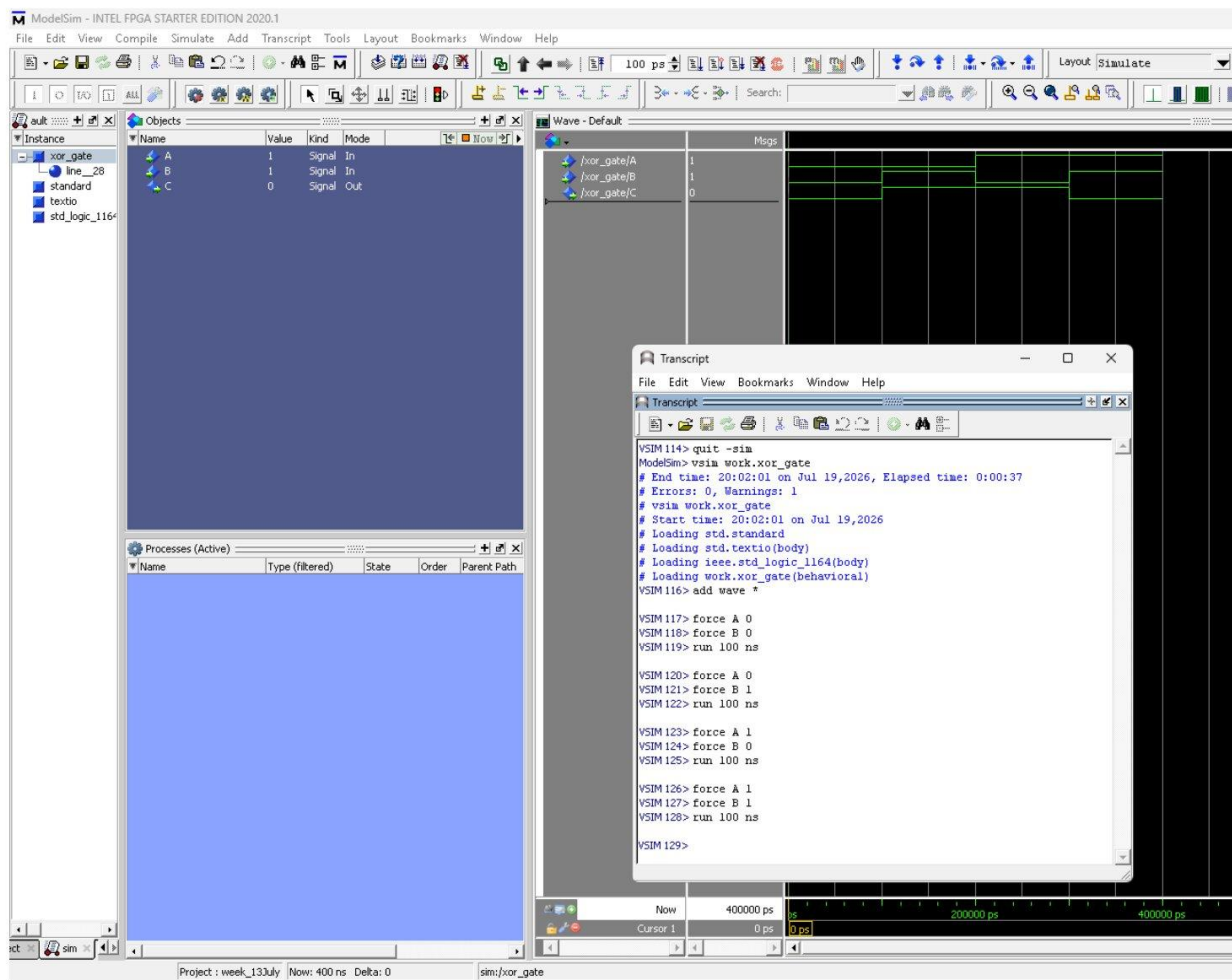
-- C gets the logical XOR of A and B

C <= A XOR B;

end Behavioral;

Circuit :





ModelSim Simulation – XOR Gate

Assignment - 1

Program - 1 - Design and implementation of an XNOR Gate using VHDL

⇒ Inputs : A, B

⇒ Output : C

⇒ Truth table :

A	B	C
0	0	1
0	1	0
1	0	0
1	1	1

⇒ Program :

Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity xnor_gate is

Port (

A : in STD_LOGIC; -- Input A

B : in STD_LOGIC; -- Input B

C : out STD_LOGIC -- Output C = A XNOR B

);

end xnor_gate;

architecture Behavioral of xnor_gate is

begin

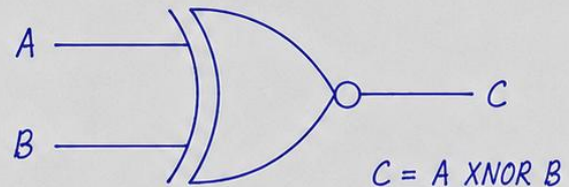
-- Concurrent signal assignment:

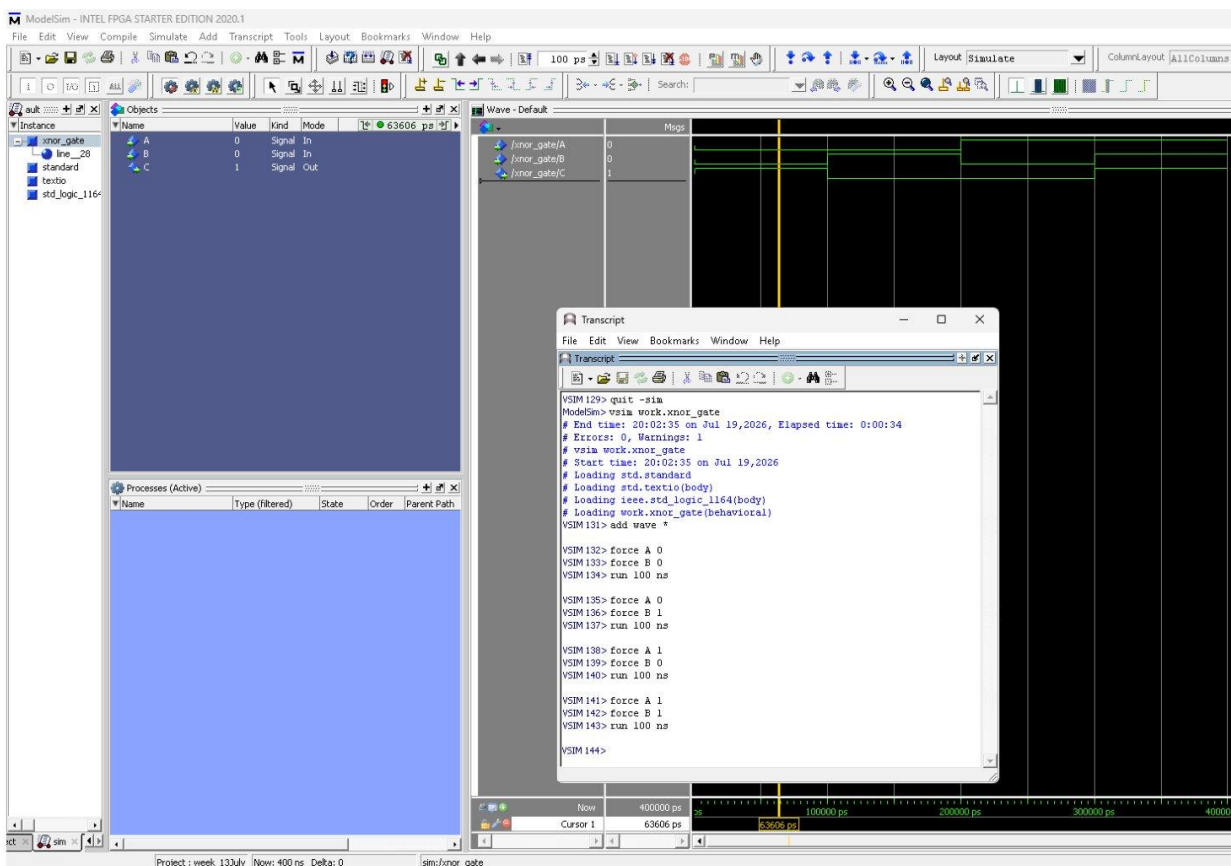
-- C gets the logical XNOR of A and B

C <= A XNOR B;

end Behavioral;

Circuit :





ModelSim Simulation – XNOR Gate